

Шпаргалка к собеседованию: Разработчик ИИ-продуктов / Ведущий программист (ИИ)

Компания: Group365 / MagnitMedia (R&D лаборатория, внутренний стартап)

Продукт: Overlay AI — система, объединяющая LLM + Symbolic AI + внешняя память

Фокус: Digital Twin (Zettelkasten 2.0), Deep Research, RAG, графы знаний, нейросимволика

Формат: удалённо, проектная работа, от 300к/мес

1. RAG (Retrieval-Augmented Generation) — КЛЮЧЕВАЯ ТЕМА

1.1 Что такое RAG и зачем

RAG решает 3 фундаментальные проблемы LLM:

- **Галлюцинации** — модель генерирует правдоподобный, но ложный текст
- **Устаревшие знания** — обучение заморожено на дате cutoff
- **Нет доступа к приватным данным** — корпоративные документы, базы

Архитектура RAG:

```
User Query → Embedding → Vector Search → Top-K chunks → LLM(query + context) → Answer
```

1.2 Этапы RAG-пайплайна

Indexing (офлайн):

1. **Document Loading** — PDF, HTML, Markdown, DB records
2. **Chunking** — разбиение на фрагменты (500-1000 токенов)
 - Fixed-size chunks (с overlap 10-20%)
 - Semantic chunking (по смыслу, через embedding similarity)
 - Recursive character splitting (LangChain default)
3. **Embedding** — преобразование текста в вектор
 - OpenAI `text-embedding-3-small` (1536 dim), `text-embedding-3-large` (3072 dim)
 - Open-source: `sentence-transformers/all-MiniLM-L6-v2`, BGE, E5
4. **Indexing** — сохранение в Vector DB

Retrieval (онлайн):

1. Query → Embedding
2. Similarity search (cosine, dot product, L2)
3. Top-K результатов (обычно k=3..10)
4. Reranking (опционально, через cross-encoder)

Generation:

- 1. Формирование промпта: system + retrieved context + user query
- 2. LLM генерирует ответ на основе контекста

1.3 Продвинутые техники RAG

Техника	Суть	Когда применять
Hybrid Search	BM25 (keyword) + Vector (semantic)	Когда нужны точные совпадения + семантика
HyDE	Генерируем гипотетический ответ, ищем по нему	Когда запрос абстрактный
Multi-Query	LLM генерирует N переформулировок запроса	Увеличивает recall
Parent-Child	Ищем по мелким чанкам, возвращаем родительский	Точность поиска + полнота контекста
Self-RAG	LLM сам решает, нужен ли retrieval	Экономия токенов
RAPTOR	Иерархическое суммирование + кластеризация	Длинные документы
Graph RAG	Извлечение сущностей → граф знаний → traversal	Связи между фактами
Corrective RAG (CRAG)	Проверка релевантности retrieved docs, fallback на web search	Повышение точности
Agentic RAG	Агент решает: искать, уточнять, или отвечать	Сложные multi-step запросы

1.4 Метрики качества RAG

- **Retrieval:**
 - **Precision@K** — доля релевантных среди top-K
 - **Recall@K** — доля найденных релевантных из всех релевантных
 - **MRR (Mean Reciprocal Rank)** — позиция первого релевантного
 - **NDCG** — учитывает порядок релевантных результатов
- **Generation:**
 - **Faithfulness** — ответ основан на контексте, а не выдуман
 - **Answer Relevancy** — ответ отвечает на вопрос
 - **Context Relevancy** — контекст релевантен вопросу
- **Фреймворки оценки:** RAGAS, DeepEval, TruLens

1.5 Пример кода RAG с нуля (без фреймворков)

```

import numpy as np
from openai import OpenAI

client = OpenAI()

# 1. Embedding
def embed(texts: list[str]) -> list[list[float]]:
    resp = client.embeddings.create(model='text-embedding-3-small', input=texts)
    return [r.embedding for r in resp.data]

# 2. Cosine similarity
def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# 3. Retrieval
def retrieve(query: str, chunks: list[str], embeddings: list, top_k=5):
    q_emb = embed([query])[0]
    scores = [(i, cosine_sim(q_emb, e)) for i, e in enumerate(embeddings)]
    scores.sort(key=lambda x: x[1], reverse=True)
    return [(chunks[i], s) for i, s in scores[:top_k]]

# 4. Generation
def rag_answer(query: str, context_chunks: list[str]) -> str:
    context = '\n---\n'.join(context_chunks)
    resp = client.chat.completions.create(
        model='gpt-4o',
        messages=[
            {'role': 'system', 'content': f'Answer based on context:\n{context}'},
            {'role': 'user', 'content': query},
        ],
    )
    return resp.choices[0].message.content

```

2. Vector Databases

2.1 Сравнение

DB	Тип	Особенности	Когда использовать
Pinecone	Managed cloud	Serverless, auto-scaling, metadata filtering	Продакшн, не хочешь управлять инфрой
Weaviate	Self-hosted / Cloud	GraphQL API, модульные vectorizers, hybrid search	Когда нужен гибкий self-hosted
ChromaDB	Embedded	Простой API, in-memory/persistent, для прототипов	MVP, локальная разработка
Qdrant	Self-hosted / Cloud	Rust, быстрый, payload filtering, gRPC	Высокая нагрузка, on-prem

DB	Тип	Особенности	Когда использовать
Redis (RediSearch)	In-memory	HNSW index, быстрый, уже есть в стеке	Когда Redis уже используется
pgvector	PostgreSQL extension	SQL + vectors, ACID, знакомый стек	Когда уже есть Postgres
FAISS	Library (Meta)	Не DB, а библиотека. Очень быстрый ANN	Оффлайн batch, исследования
Milvus	Distributed	Горизонтальное масштабирование, миллиарды векторов	Enterprise, огромные датасеты

2.2 Алгоритмы поиска

- **Brute-force (Flat)** — точный, $O(n)$, не масштабируется
- **HNSW (Hierarchical Navigable Small World)** — граф, $O(\log n)$, лучший баланс скорость/точность
- **IVF (Inverted File Index)** — кластеризация + поиск в ближайших кластерах
- **PQ (Product Quantization)** — сжатие векторов, экономия памяти

2.3 Пример: Qdrant

```
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct

client = QdrantClient(host='localhost', port=6333)

# Создание коллекции
client.create_collection(
    collection_name='docs',
    vectors_config=VectorParams(size=1536, distance=Distance.COSINE),
)

# Вставка
client.upsert(collection_name='docs', points=[
    PointStruct(id=1, vector=[0.1, 0.2, ...], payload={'text': 'chunk text',
'source': 'doc.pdf'}),
])

# Поиск
results = client.search(
    collection_name='docs',
    query_vector=[0.1, 0.2, ...],
    limit=5,
    query_filter=Filter(must=[FieldCondition(key='source',
match=MatchValue(value='doc.pdf'))]),
)
```

3. LLM: API, локальные модели, оптимизация инференса

3.1 OpenAI API / OpenRouter

```
# OpenAI
from openai import AsyncOpenAI
client = AsyncOpenAI(api_key='sk-...')

# OpenRouter (тот же формат, другой base_url)
client = AsyncOpenAI(
    api_key='or-...',
    base_url='https://openrouter.ai/api/v1',
)

response = await client.chat.completions.create(
    model='gpt-4o',
    messages=[{'role': 'user', 'content': 'Hello'}],
    tools=[...],          # function calling
    tool_choice='auto',
    temperature=0.7,
    max_tokens=4096,
)
```

3.2 Function Calling / Tool Use

```
tools = [{
    'type': 'function',
    'function': {
        'name': 'search_database',
        'description': 'Search the knowledge base',
        'parameters': {
            'type': 'object',
            'properties': {
                'query': {'type': 'string', 'description': 'Search query'},
                'limit': {'type': 'integer', 'default': 5},
            },
            'required': ['query'],
        },
    },
}]

# LLM возвращает tool_calls → ты выполняешь → отправляешь результат обратно
# Это основа агентных архитектур
```

3.3 Локальные LLM

Инструмент	Назначение
------------	------------

Инструмент	Назначение
vLLM	Высокопроизводительный inference server, PagedAttention, continuous batching
llama.cpp	CPU/GPU inference для GGUF моделей, минимальные зависимости
Ollama	Обёртка над llama.cpp, удобный CLI, Docker
TGI (HuggingFace)	Text Generation Inference, production-ready
SGLang	Structured generation, быстрее vLLM для некоторых задач

vLLM — ключевые концепции:

- **PagedAttention** — управление KV-cache как страницами памяти (как в ОС), экономия GPU RAM
- **Continuous Batching** — новые запросы добавляются в batch не дожидаясь завершения текущих
- **Tensor Parallelism** — распределение модели по нескольким GPU
- **Quantization** — AWQ, GPTQ, FP8 — уменьшение размера модели с минимальной потерей качества

```
# Запуск vLLM
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-70B-Instruct \
  --tensor-parallel-size 2 \
  --gpu-memory-utilization 0.9 \
  --max-model-len 8192
# Совместим с OpenAI API format
```

3.4 Оптимизация инференса

- **Batching** — группировка запросов для GPU utilization
- **KV-Cache** — кэширование ключей/значений attention для авторегрессии
- **Speculative Decoding** — маленькая модель генерирует draft, большая верифицирует
- **Quantization** — INT8/INT4/FP8 снижает memory footprint в 2-4x
- **Flash Attention** — оптимизированный attention kernel, O(n) memory вместо O(n²)
- **Prefix Caching** — кэширование общих system prompt prefix

4. AI Agents — Агентные архитектуры

4.1 Что такое AI Agent

Agent = LLM + Tools + Memory + Planning loop

```
while not done:
    action = LLM.decide(observation, memory, tools)
    if action == 'final_answer':
        return answer
    result = execute_tool(action)
    memory.add(result)
```

4.2 Паттерны агентов

Паттерн	Описание	Пример
ReAct	Reason + Act: LLM чередует рассуждение и действие	Базовый агент с tools
Plan-and-Execute	Сначала план, потом выполнение по шагам	Сложные multi-step задачи
Reflection	Агент критикует свой результат и улучшает	Код-генерация, writing
Multi-Agent	Несколько агентов с разными ролями	CrewAI, AutoGen
Supervisor	Один агент-координатор распределяет задачи	Оркестрация команды агентов
Tool-use loop	LLM вызывает tools итеративно до получения ответа	OpenAI function calling loop

4.3 LangGraph

LangGraph — библиотека для построения stateful, multi-step агентов как графов.

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, Annotated
import operator

class AgentState(TypedDict):
    messages: Annotated[list, operator.add]
    next_step: str

def agent_node(state: AgentState):
    # LLM решает что делать
    response = llm.invoke(state['messages'])
    return {'messages': [response]}

def tool_node(state: AgentState):
    # Выполняем tool calls
    last_msg = state['messages'][-1]
    results = execute_tools(last_msg.tool_calls)
    return {'messages': results}

def should_continue(state: AgentState):
    last_msg = state['messages'][-1]
    if last_msg.tool_calls:
        return 'tools'
    return END

# Граф
graph = StateGraph(AgentState)
```

```

graph.add_node('agent', agent_node)
graph.add_node('tools', tool_node)
graph.set_entry_point('agent')
graph.add_conditional_edges('agent', should_continue, {'tools': 'tools', END:
END})
graph.add_edge('tools', 'agent')

app = graph.compile()
result = app.invoke({'messages': [HumanMessage(content='...')]}))

```

Ключевые концепции LangGraph:

- **State** — типизированное состояние, передаётся между нодами
- **Nodes** — функции, обрабатывающие state
- **Edges** — связи между нодами (conditional / unconditional)
- **Checkpointing** — сохранение состояния для resume / human-in-the-loop
- **Subgraphs** — вложенные графы для модульности

4.4 Проектирование агента без фреймворков (важно для этой вакансии!)

```

import json
from openai import AsyncOpenAI

class Agent:
    def __init__(self, client: AsyncOpenAI, tools: list[dict], system_prompt:
str):
        self.client = client
        self.tools = tools
        self.system_prompt = system_prompt
        self.tool_registry = {} # name -> callable

    def register_tool(self, name: str, fn):
        self.tool_registry[name] = fn

    async def run(self, user_message: str, max_iterations: int = 10) -> str:
        messages = [
            {'role': 'system', 'content': self.system_prompt},
            {'role': 'user', 'content': user_message},
        ]

        for _ in range(max_iterations):
            response = await self.client.chat.completions.create(
                model='gpt-4o',
                messages=messages,
                tools=self.tools,
                tool_choice='auto',
            )
            msg = response.choices[0].message

            if not msg.tool_calls:
                return msg.content

```



```
# Добавляем assistant message с tool_calls
messages.append(msg)

# Выполняем каждый tool call
for tc in msg.tool_calls:
    fn = self.tool_registry.get(tc.function.name)
    args = json.loads(tc.function.arguments)
    result = await fn(**args) if fn else {'error': 'Unknown tool'}
    messages.append({
        'role': 'tool',
        'tool_call_id': tc.id,
        'content': json.dumps(result, ensure_ascii=False),
    })

return 'Max iterations reached'
```

5. Python Expert — Ключевые темы

5.1 Асинхронность (asyncio)

```
import asyncio
import aiohttp

# Event loop, coroutines, tasks
async def fetch(url: str) -> str:
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as resp:
            return await resp.text()

# Параллельное выполнение
results = await asyncio.gather(
    fetch('https://api1.com'),
    fetch('https://api2.com'),
    fetch('https://api3.com'),
)

# Semaphore для ограничения concurrency
sem = asyncio.Semaphore(10)
async def limited_fetch(url):
    async with sem:
        return await fetch(url)

# Queue для producer-consumer
queue = asyncio.Queue(maxsize=100)

async def producer():
    for item in items:
        await queue.put(item)
```

```

async def consumer():
    while True:
        item = await queue.get()
        await process(item)
        queue.task_done()

```

Ключевые концепции:

- `await` — приостанавливает корутину, отдаёт управление event loop
- `asyncio.gather()` — параллельный запуск, ждёт все
- `asyncio.create_task()` — запуск в фоне
- `asyncio.Semaphore` — ограничение concurrency
- `asyncio.Queue` — потокобезопасная очередь
- **НЕ блокировать event loop** — CPU-bound задачи в `run_in_executor()`

5.2 Структуры данных и алгоритмическая сложность

Операция	list	dict	set	deque
Доступ по индексу	$O(1)$	—	—	$O(n)$
Поиск элемента	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Вставка в конец	$O(1)^*$	$O(1)$	$O(1)$	$O(1)$
Вставка в начало	$O(n)$	—	—	$O(1)$
Удаление	$O(n)$	$O(1)$	$O(1)$	$O(1)/O(n)$

* amortized

Важные структуры:

- `collections.defaultdict` — dict с дефолтным значением
- `collections.Counter` — подсчёт элементов
- `heapq` — min-heap, приоритетная очередь $O(\log n)$
- `functools.lru_cache` — мемоизация
- `dataclasses` / `pydantic.BaseModel` — типизированные модели

5.3 Паттерны проектирования (Python-specific)

- **Dependency Injection** — передача зависимостей через конструктор
- **Repository Pattern** — абстракция доступа к данным
- **Strategy** — подмена алгоритма через callable/class
- **Observer** — event-driven (asyncio events, callbacks)
- **Factory** — создание объектов по параметрам
- **Singleton** — через module-level instance (Pythonic way)

5.4 Type Hints и Pydantic

```
from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime

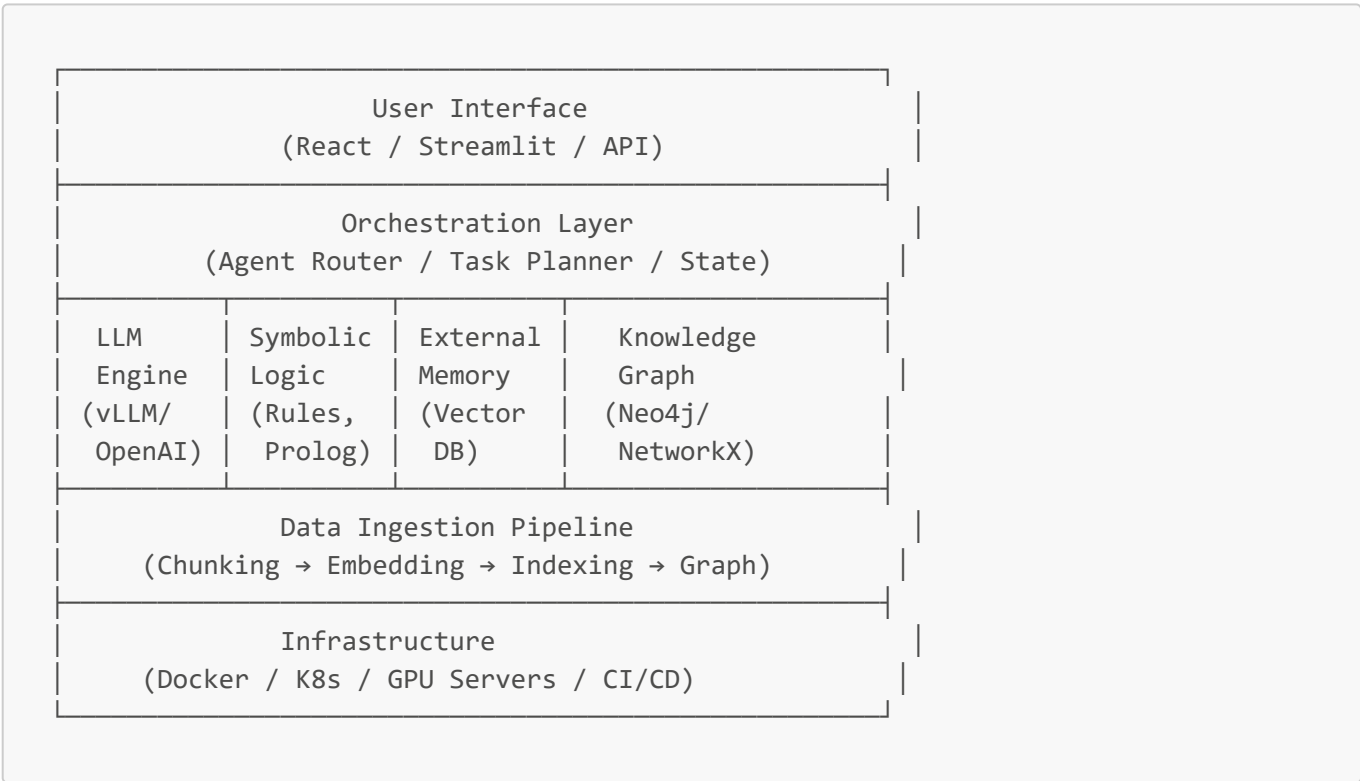
class Document(BaseModel):
    id: str
    content: str
    metadata: dict[str, str] = Field(default_factory=dict)
    embedding: Optional[list[float]] = None
    created_at: datetime = Field(default_factory=datetime.now)

class Config:
    json_schema_extra = {'example': {'id': 'doc1', 'content': 'Hello'}}

# Валидация автоматическая
doc = Document(id='1', content='test')
doc_json = doc.model_dump_json()
doc_back = Document.model_validate_json(doc_json)
```

6. System Design & Architecture

6.1 Архитектура Overlay AI (то, что строит компания)



6.2 Microservices vs Monolith

Для AI-продукта на ранней стадии (стартап):

- Начинать с **modular monolith** — один сервис, но чёткие модули
- Выделять в микросервисы по мере роста: inference server, ingestion pipeline, API gateway

- **Event-driven** для асинхронных задач (ingestion, embedding)

6.3 Event-Driven Architecture

Producer → Message Broker → Consumer
(Redis Streams / RabbitMQ / Kafka)

Примеры событий в AI-системе:

- document.uploaded → chunking → embedding → indexing
- query.received → retrieval → generation → response
- feedback.received → reranking model update

6.4 Как проектировать RAG-систему (вопрос на собесед)

Вопрос: «Спроектируй систему Deep Research для корпоративных документов»

Ответ по шагам:

1. Ingestion Pipeline:

- Загрузка документов (PDF, DOCX, HTML) → парсинг → chunking
- Embedding (batch processing, GPU) → Vector DB
- Entity extraction → Knowledge Graph (Neo4j)
- Metadata indexing → PostgreSQL

2. Query Pipeline:

- Query understanding (intent classification, query expansion)
- Hybrid retrieval: Vector search + BM25 + Graph traversal
- Reranking (cross-encoder)
- Multi-hop reasoning (если нужно собрать инфо из нескольких документов)

3. Generation:

- Промпт с контекстом + source citations
- Streaming response
- Confidence scoring

4. Feedback Loop:

- Thumbs up/down → fine-tune reranker
- Usage analytics → improve chunking strategy

7. Docker & DevOps

7.1 Docker для AI-проектов

```
# Multi-stage build
FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.12/site-packages
/usr/local/lib/python3.12/site-packages
COPY . .
CMD ["python", "-m", "app"]
```

```
# docker-compose.yml для AI-стека
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - QDRANT_URL=http://qdrant:6333
    depends_on: [qdrant, postgres]

  qdrant:
    image: qdrant/qdrant:latest
    ports: ["6333:6333"]
    volumes: ["qdrant_data:/qdrant/storage"]

  postgres:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_DB: knowledge
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes: ["pg_data:/var/lib/postgresql/data"]

volumes:
  qdrant_data:
  pg_data:
```

7.2 GPU в Docker

```
# NVIDIA Container Toolkit
docker run --gpus all -p 8000:8000 vllm/vllm-openai:latest \
  --model meta-llama/Llama-3.1-8B-Instruct

# docker-compose c GPU
services:
  vllm:
```

```

image: vllm/vllm-openai:latest
deploy:
  resources:
    reservations:
      devices:
        - driver: nvidia
          count: all
          capabilities: [gpu]

```

7.3 CI/CD

```

# GitHub Actions пример
name: Deploy
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build & Push Docker
        run: |
          docker build -t registry.example.com/app:${{ github.sha }} .
          docker push registry.example.com/app:${{ github.sha }}
      - name: Deploy
        run: ssh server "docker pull ... && docker-compose up -d"

```

8. Knowledge Graphs & Symbolic AI

8.1 Зачем графы знаний в AI

- LLM плохо работает с **связями между фактами**
- Граф хранит: (Entity) -[Relation]-> (Entity)
- Пример: (Иванов) -[работает_в]-> (Компания X) -[производит]-> (Продукт Y)
- Позволяет **multi-hop reasoning**: «Кто производит продукты, с которыми работает Иванов?»

8.2 Neo4j + LLM

```

from neo4j import GraphDatabase

driver = GraphDatabase.driver('bolt://localhost:7687', auth=('neo4j', 'password'))

# Создание сущностей из текста (LLM extraction)
def create_entity(tx, name, type):
    tx.run('MERGE (e:Entity {name: $name, type: $type})', name=name, type=type)

```

```
def create_relation(tx, from_name, to_name, relation):
    tx.run('''
        MATCH (a:Entity {name: $from}), (b:Entity {name: $to})
        MERGE (a)-[r:RELATES {type: $rel}]->(b)
    ''', **{'from': from_name, 'to': to_name, 'rel': relation})

# Graph RAG: поиск по графу + LLM
def graph_rag_query(question: str):
    # 1. LLM извлекает сущности из вопроса
    entities = llm_extract_entities(question)
    # 2. Cypher query для получения связанных фактов
    facts = graph_query(entities, depth=2)
    # 3. LLM генерирует ответ на основе фактов
    return llm_answer(question, facts)
```

8.3 Zettelkasten 2.0 (Digital Twin) — концепция компании

Zettelkasten — метод управления знаниями (атомарные заметки + связи).

Digital Twin в контексте AI:

- Каждый документ/факт — нода в графе
- Связи извлекаются автоматически (LLM + NER)
- Поиск через комбинацию: vector similarity + graph traversal + symbolic rules
- «Цифровой двойник» знаний организации

9. mem0 и Cognee (упомянуты в вакансии)

9.1 mem0 — Memory Layer for AI

```
from mem0 import Memory

m = Memory()

# Добавление памяти
m.add('Пользователь предпочитает Python', user_id='user1')
m.add('Работает в финтехе', user_id='user1')

# Поиск релевантной памяти
results = m.search('Какой язык предпочитает?', user_id='user1')
# → [{'memory': 'Пользователь предпочитает Python', 'score': 0.95}]
```

Суть: Персистентная память для AI-агентов. Хранит факты о пользователях, автоматически обновляет/мержит противоречивую информацию.

9.2 Cognee — Knowledge Management for AI

Cognee автоматически строит knowledge graph из документов:

- Ingestion → Chunking → Entity/Relation extraction → Graph
 - Поддерживает RAG поверх графа
 - Интеграция с LLM для query understanding
-

10. n8n — Автоматизация

n8n — open-source workflow automation (аналог Zapier).

Релевантность для вакансии:

- Понимание принципов: nodes, triggers, webhooks, data flow
 - Компания хочет, чтобы ты мог спроектировать **более эффективную архитектуру с нуля**
 - Ключевой поинт: n8n хорош для простых workflow, но для AI-агентов нужна кастомная архитектура с:
 - Stateful execution
 - Conditional branching на основе LLM output
 - Streaming
 - Error recovery и retry logic
-

11. Frontend (плюс, не обязательно)

Упомянуты: React, Vue, Streamlit.

Streamlit — быстрый прототип:

```
import streamlit as st

st.title('AI Knowledge Base')
query = st.text_input('Ask a question:')
if query:
    with st.spinner('Searching...'):
        answer = rag_pipeline(query)
    st.markdown(answer)
```

React — для продакшн UI:

- Ты уже делал React-админку (KommoMCP) — это отличный пример для собеседа
-

12. Вопросы, которые могут задать

Технические

1. «**Расскажи про свой опыт с RAG**» → KommoMCP: RAG-based tool retrieval, dynamic prompt generation, YAML tool definitions → embedding → cosine similarity → top-K tools → LLM prompt
2. «**Как бы ты спроектировал систему Deep Research?**» → См. раздел 6.4

- 3. «**Чем отличается RAG от fine-tuning?**» → RAG: внешние знания в runtime, дешевле, обновляемо. Fine-tuning: знания в весах модели, дороже, нужен retrain. Для частных данных — RAG. Для стиля/формата — fine-tuning.
- 4. «**Как решить проблему галлюцинаций?**» → RAG с source citations, Corrective RAG, confidence scoring, symbolic verification rules, human-in-the-loop
- 5. «**Расскажи про агентные архитектуры**» → ReAct, Plan-and-Execute, Multi-Agent. KommoMCP: iterative tool-calling loop (до 10 итераций), LLM сам выбирает tools.
- 6. «**Как оптимизировать инференс локальных LLM?**» → vLLM (PagedAttention, continuous batching), quantization (AWQ/GPTQ), Flash Attention, prefix caching, speculative decoding
- 7. «**Docker + GPU: как настроить?**» → NVIDIA Container Toolkit, `--gpus all`, docker-compose с `deploy.resources.reservations.devices`

Архитектурные

- 8. «**Microservices vs Monolith для AI-стартапа?**» → Modular monolith на старте, выделять сервисы по мере роста. Inference server — первый кандидат на выделение.
- 9. «**Event-driven vs Request-response?**» → Event-driven для async задач (ingestion, embedding). Request-response для синхронных API. Комбинация обоих.
- 10. «**Как масштабировать RAG-систему?**» → Horizontal scaling Vector DB (Milvus/Qdrant cluster), кэширование embeddings, async ingestion pipeline, CDN для static assets, read replicas для metadata DB

Поведенческие

- 11. «**Расскажи про проект с нуля**» → KommoMCP: от идеи до продакшна. Multi-tenant SaaS, AI-агент для CRM, RAG, monitoring dashboard, React admin panel. 5 пользователей, 3 активных CRM.
- 12. «**Как декомпозируешь задачи?**» → Разбиваю на вертикальные слайсы (каждый — рабочий инкремент). Приоритизация по бизнес-ценности. Для junior/middle — чёткие спецификации с примерами.

13. Твои козыри для этого собеседования

KommoMCP — идеальный кейс

Требование вакансии	Твой опыт в KommoMCP
RAG-архитектуры	RAG-based tool retrieval, dynamic prompts
AI Agents	Iterative tool-calling loop, 10+ tools
Python Expert	asyncio, aiohttp, Pydantic, type hints
Fullstack	React admin + Python backend
Docker, DevOps	Docker deployment, systemd, nginx, CI

Требование вакансии	Твой опыт в KomtoMCP
MVP с нуля	Полный цикл: архитектура → код → деплой
Multi-tenant SaaS	Tenant isolation, orchestration
Мониторинг	Interaction logger, admin dashboard

Ключевые фразы для собеса

- «Я строил RAG-систему без фреймворков — YAML tool definitions, embedding-based retrieval, dynamic prompt assembly»
- «Агентный цикл с iterative tool calling — LLM сам выбирает какие API вызвать, до 10 итераций»
- «Multi-tenant архитектура с изоляцией данных, оркестрацией инфраструктуры»
- «Полный цикл от проектирования до продакшна — бот обслуживает реальных пользователей»
- «React-админка для мониторинга: юзеры, CRM-подключения, AI-сессии в реальном времени»

14. Вопросы, которые стоит задать ИМ

1. «Какой объём данных планируется обрабатывать? Это влияет на выбор Vector DB и стратегию chunking»
2. «Есть ли уже прототип или начинаем с чистого листа?»
3. «Какие LLM планируете использовать — только облачные или есть GPU для локальных?»
4. «Как устроена текущая команда? Сколько разработчиков?»
5. «Какие метрики успеха продукта на ближайшие 3 месяца?»
6. «Symbolic AI часть — это rule engine, Prolog, или что-то кастомное?»
7. «Есть ли уже датасеты для тестирования качества RAG?»